

Engenharia da Computação

TecHacker- Relatório PI

Endpoint Ivestigator: Ferramenta de Análise de Vulnerabilidade

Silas André Pereira de Brito

Yamandú Germano Cavalcanti

Professor: Rodolfo Avelino

05/10/2026

1) Problema e Objetivo

O Endpoint Ivestigator investiga um snapshot de um Endpoint GNU/Linux correlacionando processos, permissões, serviços e registros de log. O objetivo não é declarar automaticamente que uma máquina foi comprometida, mas organizar evidências, explicitar seu contexto e indicar hipóteses para verificação.

Problema: Disponibilizar a um usuário comum informações de vulnerabilidades em sua máquina de forma intuitiva por meio de leituras aos processos e outras seções importantes dentro do computador.

Objetivo: Coletar, normalizar, correlacionar e reportar observações de segurança de forma explicável, incluindo resultados inconclusivos

2) Estratégia de investigação

A ferramenta opera sob demanda e pode receber um dataset, coletar um snapshot live em Linux ou executar um gerador de dados sintéticos. As quatro fontes normalizadas são:

FONTE	INFORMACAO	USO NA INVESTIGAÇÃO
processes.csv /proc	PID, PPID, usuário, estado e comando	Identidade, ancestralidade e contexto de execução
permissions.csv / stat	arquivo, dono, grupo, modo e mtime	Capacidade de modificação do recurso utilizado

services.txt / systemctl	unit, estado, usuário e EXECSTART	Associação entre serviço, processo e recurso
journal.log / logs do sistema	data, origem, PID e mensagem	Reconstrução temporal e consistência

3) Arquitetura implementada

Foi escolhido desenvolver em Python pela facilidade de execução em Linux e escalabilidade/adaptabilidade

Entrada e normalização

- collectors/dataset.py: parsers CSV, TXT e journal.
- collectors/live.py: leitura contextual de /proc, systemctl, stat e logs.
- collectors/custom.py: contrato para scripts geradores.
- models.py: Process, FileMeta, Service, LogEvent, Snapshot e Finding

Análise e saída

- correlators/: mapa de serviços, permissões, contexto, configurações e timeline.
- report.py: relatório textual e findings em JSON.
- graph.py: mapa de relações em SVG, JSON ou DOT.
- investigator.py: CLI e orquestração do pipeline

4) Principais correlações

CORRELAÇÃO	REGRA IMPLEMENTADA	RESULTADO
C1 Serviço + arquivo + usuario	Serviço root usa recurso modificável por não privilegiado; considera arquivo, diretório pai e mtime.	Possível relação de privilégio insegura; não prova exploração.
C2 Processo + PPID + usuário	Constrói ancestralidade e verifica origem, identidade, caminho executado e evidencias de elevação	Contexto de execução, com contra evidências quando aplicável.
C3 Processo + serviço + log	Ordena eventos, processos relevantes e início de serviços; verifica divergência PID/origem.	Linha do tempo e findings de consistência.
Complementar	Permissão ampla sem vínculo observado com execução privilegiada	inconclusivo ou informativo, reduzindo o índice de falso positivo

5) Como a ferramenta separa Fato e Interpretação

Cada Finding possui quatro camadas. O formato impede que um indicador isolado seja apresentado como conclusão:

CAMADA	EXEMPLO NO PROJETO
Evidência	/opt/reports/report.sh possui modo 0777 e não aparece referenciado por serviço/processo no snapshot.
Interpretação	O arquivo pode ser modificado por qualquer usuário, mas não foi observado uso privilegiado.
Hipótese	Se um job privilegiado executar o arquivo, pode existir risco de escalonamento.
Evidência Ausente	Unit, cron, timer, auditd, hash e histórico de modificação que confirmariam ou rejeitariam a hipótese.

Regra de interpretação: serviço root isolado, nome suspeito ou permissão ampla isolada não são classificados automaticamente como vulnerabilidade. A severidade representa prioridade de investigação, não veredito de comprometimento

6) Uso de IA e Decisões Técnicas

- Foi utilizado apoio de IA para alguns aspectos do projeto: arquitetura de algumas partes e módulos do projeto, entendimento e pesquisa sobre alguns tópicos necessários e preenchimento de código com o Inline Suggestions
- Foi projeto diversos tipos de saídas e uso de flags para maximizar a possibilidades de saídas e melhorar a interpretação do usuário provocando a análise diversos aspectos.
- O foco foi mais verticalizado, buscamos entregar um bom detalhamento das correlações escolhidas e não exploramos tantas outras conexões, muito por falta de tempo
- A linguagem principal é o python pela sua versatilidade e boas bibliotecas que possibilitaram uma boa leitura de dados sem um tempo de execução tão custoso, mas para sistemas mais pesados linguagens mais rápidas podem trazer um melhor aspecto

7) Limitações

- Snapshot único: não observa evolução temporal nem histórico completo (caso Postgres do vídeo de Demonstração)
- O vínculo serviço-arquivo é heurístico e pode produzir falsos positivos ou negativos.
- Não observamos alguns aspectos que poderiam refinar/ampliar nossas conclusões, SUID/SGID, atributos, crontabs, units completas, rede e memória.
- Logs curtos e ausência de evento não provam ausência de atividade.
- Sem hash, conteúdo ou auditd, não se confirma que um arquivo foi alterado ou executado.
- No modo live sem root, a visão de processos e arquivos pode ser parcial, mas a própria execução do Python por meio de um “sudo” já gera um caso falso positivo de alta vulnerabilidade

8) Entregáveis

ITEM	REFERÊNCIA
Código-fonte / local para uso da aplicação (Versionamos o Código em um repo no GitHub).	https://github.com/YamanduGermano/Endpoint-Investigator
README com arquitetura, instruções de execução, fontes, correlações e limitações	https://github.com/YamanduGermano/Endpoint-Investigator/blob/main/README.md
Relatório técnico de até 4 páginas	Este Documento
Demonstração Prática em Vídeo	https://youtu.be/GRqNC8julws